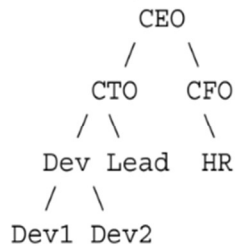


Tree Assignment

Problem 1: Tree Terminology Identification

Scenario: You are developing a company organizational chart application. The company has the following hierarchy:



Questions: a) Identify all leaf nodes in this organizational tree. b) What is the height of the tree? c) What is the depth of “Dev Lead”? d) List all ancestors of “Dev1”. e) What is the degree of the CTO node?

Sol:-

a) Leaf nodes = nodes with no children

From this tree – Dev1, Dev2, HR, Lead

b) Height = no of edges in the longest path from root to leaf, so the height of this tree is 3.

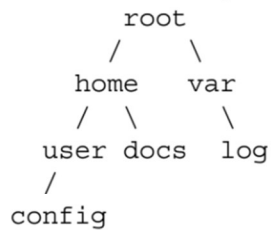
c) Depth = number of edges from root to that node, for this the depth is 2.

d) Ancestors = all nodes from root to the parent of the node, so ancestors of Dev1 are Dev, CTO, CEO.

e) Degree of CTO = Children of CTO = Dev, Lead → 2 children

Problem 2: Traversal Application

Scenario: A file system is represented as a tree structure:



You need to implement different operations:

Questions: a) If you want to list all files in alphabetical order, which traversal should you use if this were a BST? b) To calculate the total size of a directory (sum of all files in subdirectories), which traversal is most appropriate? c) To create a backup by copying the entire directory structure starting from the root, which traversal would you use? d) Perform inorder, preorder, and postorder traversals on the above tree. e) Explain how postorder traversal can be used to safely delete the entire directory structure.

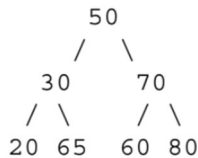
Sol:-

- a) Inorder Traversal (Left → Root → Right)
- b) To calculate size, you must visit all children first then combine their sizes at the parent Postorder Traversal (Left → Right → Root)
- c) Preorder Traversal (Root → Left → Right)
- d) Inorder – config, user, home, docs, root, var, log
Preorder – root, home, user, config, docs, var, log
Postorder – config, user, docs, home, log, var, root
- e) Postorder = children first, then parent, we cannot delete a folder before deleting its contents otherwise, we lose access to child files

Problem 3: BST Construction and Validation

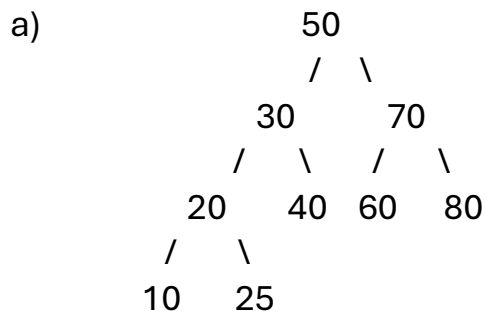
Scenario: A library management system needs to maintain books sorted by their ISBN numbers for quick lookups. The following ISBN numbers are to be inserted into a BST in the given order: 50, 30, 70, 20, 40, 60, 80, 10, 25.

Questions: a) Draw the resulting BST after all insertions. b) Show the sequence of comparisons made when searching for ISBN 25. c) Perform inorder traversal on your BST and verify the result is sorted. d) Is the following tree a valid BST? Explain why or why not:



e) What would be the height of a completely skewed BST with the same 9 elements? Compare with your balanced tree's height.

Sol :-



b) inorder traversal -

10 → 20 → 25 → 30 → 40 → 50 → 60 → 70 → 80

result is sorted, so BST property holds

c) validity of given tree

given tree has 65 as right child of 30, but $65 > 50$ and lies in left subtree of 50. This violates BST rule (all nodes in left subtree must be < 50) therefore not a valid BST

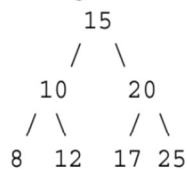
d) completely skewed BST height = 8

balanced BST with 9 nodes height = 3

skewed tree height much larger than balanced tree height

Problem 4: BST Operations

Scenario: A student attendance system stores student roll numbers in a BST for efficient searching. The current BST contains: 15, 10, 20, 8, 12, 17, 25.



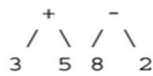
Questions: a) Show step-by-step how to delete node 10 from this BST (which has two children). b) After deletion in part (a), insert roll number 14 into the modified tree. Show the steps. c) A new student with roll number 9 joins. Show the insertion process. d) If you need to find all students with roll numbers between 10 and 20, which traversal would be most efficient and why? e) Calculate the time complexity for searching roll number 25 in both best-case and worst-case scenarios.

Sol :-

- a) step 1: find inorder successor (smallest in right subtree) → 12
step 2: replace 10 with 12
step 3: delete original 12 node
- b) $14 < 15 \rightarrow$ go left
 $14 > 12 \rightarrow$ go right → insert
- c) $9 < 15 \rightarrow$ left
 $9 < 12 \rightarrow$ left
 $9 > 8 \rightarrow$ right → insert
- d) use inorder traversal (left → root → right)
because it gives sorted order and allows easy range filtering
- e) best case: $O(1)$ (if found at root, not here but theoretical best case)
worst case (skewed tree): $O(n)$

Problem 5: Expression Tree Evaluation

Scenario: A calculator application uses expression trees to evaluate mathematical expressions. Consider the expression: $(3 + 5) \times (8 - 2)$
The expression tree is:



Questions: a) Perform postorder traversal on this tree and explain how it represents the postfix notation of the expression. b) Perform inorder traversal and show how adding parentheses gives the original infix expression. c) Perform preorder traversal and explain how it represents the prefix notation. d) If you were to evaluate this expression using postorder traversal, describe the algorithm (using a stack) that would compute the final result. e) Create an expression tree for: $a \times b + c / d - e$, and perform all three traversals (inorder, preorder, postorder).

Sol :-

a) $3\ 5 + 8\ 2 - *$

this is postfix notation: operands first, then operator

b) $(3 + 5) * (8 - 2)$

c) $* + 3\ 5 - 8\ 2$

d) postfix: $3\ 5 + 8\ 2 - *$

e) step 1: push 3

step 2: push 5

step 3: $+$ $\rightarrow$ pop 5,3 $\rightarrow$ compute $3+5=8$ $\rightarrow$ push 8

step 4: push 8

step 5: push 2

step 6: $-$ $\rightarrow$ pop 2,8 $\rightarrow$ compute $8-2=6$ $\rightarrow$ push 6

step 7: $*$ $\rightarrow$ pop 6,8 $\rightarrow$ compute $8*6=48$ $\rightarrow$ push 48

result = 48